

Roteiro de Aula e Atividade Prática

Projeto de modelagem e desenvolvimento de banco de dados

ENTREGA FINAL

Integrantes

- Yasmin Silva Dias
 - Ana Clara
 - Evilyn Rossin
 - Gabriel Alves
- Ryan Rodrigues

Tema Escolhido

Sistema de Biblioteca

Requisitos do Sistema

- Cadastrar autores.
- Cadastrar livros.
- Cadastrar usuários.
- Registrar empréstimos de livros.
- Consultar informações dos livros e empréstimos.

Versão: Aluno

Componente: Modelagem e Desenvolvimento de Banco de Dados / DDL x DML

Tema: normalização, UML, análise de dados com SQL e projeto final de banco de dados.

Duração sugerida: 50 minutos de aula teórica + 50 minutos de atividade prática.

Organização: grupos de 3 a 5 estudantes, com divisão simples de papéis.

1. Objetivo geral

Explicar, de forma simples e aplicada, como planejar, modelar, criar, testar e documentar um banco de dados relacional. A proposta é mostrar que um banco de dados não começa diretamente no código SQL: primeiro é necessário entender o problema, identificar os dados, organizar as tabelas, reduzir repetições, criar relacionamentos e validar se as consultas funcionam.

2. Habilidades trabalhadas

- Identificar entidades, atributos e relacionamentos de um sistema simples.
- Aplicar noções básicas de normalização para reduzir redundâncias.
- Representar a estrutura do banco com tabela de modelagem, diagrama ER ou diagrama de classes simplificado.
- Criar tabelas usando comandos DDL, inserir dados usando DML e realizar consultas SQL simples.
- Organizar um relatório final com requisitos, estrutura, scripts e resultados de consultas.

3. Conteúdo teórico com analogias

Banco de dados relacional

Definição simples: É uma forma organizada de armazenar dados em tabelas relacionadas.

Analogia: Imagine um armário com várias gavetas. Cada gaveta guarda um tipo de coisa, e tudo fica mais fácil de encontrar.

Tabela

Definição simples: Estrutura que guarda dados sobre um assunto específico, como Clientes, Produtos ou Pedidos.

Analogia: É como uma planilha: colunas representam informações e linhas representam registros.

Entidade

Definição simples: Algo importante do sistema sobre o qual precisamos guardar informações.

Analogia: É uma gaveta principal do armário. Se o sistema precisa guardar clientes, Cliente vira uma entidade.

Atributo

Definição simples: Uma característica da entidade, como nome, email, telefone ou data.

Analogia: É uma etiqueta dentro da ficha. Na ficha do cliente, existem etiquetas como nome e telefone.

Chave primária

Definição simples: Campo que identifica cada registro de forma única.

Analogia: É como o RG ou CPF daquele registro: não deve se repetir.

Chave estrangeira

Definição simples: Campo usado para ligar uma tabela a outra.

Analogia: É como colocar o número do RG do cliente na ficha do pedido para saber quem fez aquele pedido.

Relacionamento

Definição simples: Ligação entre entidades, como Cliente faz Pedido ou Pedido contém Produto.

Analogia: É uma ponte entre duas gavetas do armário.

Normalização

Definição simples: Processo de organizar tabelas para evitar repetição, bagunça e inconsistência.

Analogia: É como arrumar uma mochila bagunçada: caderno em um lugar, estojo em outro e folhas separadas por matéria.

Por que é importante: evita que a mesma informação seja repetida em vários lugares, reduz erros de atualização e facilita a manutenção.

DDL

Definição simples: Comandos usados para criar ou alterar a estrutura do banco, como CREATE TABLE.

Analogia: É a construção da casa: paredes, cômodos e portas.

Exemplo: CREATE TABLE Cliente (...) cria a estrutura da tabela Cliente.

DML

Definição simples: Comandos usados para manipular dados, como INSERT, UPDATE, DELETE e SELECT.

Analogia: É colocar, mexer ou consultar os móveis dentro da casa.

Exemplo: INSERT INTO Cliente (...) adiciona um cliente dentro da tabela.

Consulta SQL

Definição simples: Pergunta feita ao banco para obter uma resposta.

Analogia: É como perguntar ao armário: quais produtos tenho? Qual cliente comprou mais?

Funções analíticas

Definição simples: Funções usadas para comparar linhas, criar rankings e analisar padrões.

Analogia: É uma lupa que ajuda a enxergar ordem, comparação e evolução nos dados.

Documentação

Definição simples: Registro organizado do que foi planejado, criado e testado.

Analogia: É o manual de instruções do banco para outra pessoa conseguir entender e manter no futuro.

3.1 Quadro reforçado de analogias para explicar os conceitos

Use este quadro como apoio para explicar os conceitos sem começar pela linguagem técnica. A ideia é apresentar primeiro a imagem mental e depois ligar ao termo de banco de dados.

Conceito	Analogia para explicar em sala	Frase simples para o aluno
Banco de dados relacional	Armário com gavetas etiquetadas.	Cada tabela é uma gaveta; se tudo fica separado e identificado, achamos a informação rápido.
Entidade	Uma categoria principal de ficha.	Se eu preciso guardar informações sobre clientes, Cliente vira uma entidade.
Atributo	Campos de uma ficha cadastral.	Nome, telefone e e-mail são os espaços que preenchemos na ficha.
Registro	Uma ficha preenchida.	Cada linha da tabela representa um

		item real, como um cliente específico.
Chave primária	CPF/RG da ficha.	Serve para identificar um registro sem confundir com outro.
Chave estrangeira	Número de referência em outra ficha.	No pedido, coloco o id do cliente para saber quem fez a compra.
Relacionamento	Ligação entre fichas.	Cliente se liga a Pedido; Produto se liga a ItemPedido.
Normalização	Arrumar mochila bagunçada.	Tiramos repetição e colocamos cada informação no lugar certo.
1NF	Uma coisa por compartimento.	Não colocamos vários telefones na mesma célula.
2NF	Cada informação na gaveta que realmente pertence.	Dados do cliente ficam em Cliente, não repetidos em Pedido.
3NF	Evitar informação dependente de outra informação comum.	Se cidade depende do CEP, podemos separar CEP/Cidade em outra tabela.
4NF	Separar listas independentes.	Habilidades e projetos de um funcionário não devem ficar misturados na mesma coluna.
DDL	Construir a casa.	CREATE TABLE cria a estrutura do banco.
DML	Usar a casa depois de pronta.	INSERT, UPDATE, DELETE e SELECT mexem nos dados.
JOIN	Juntar duas fichas pelo número de referência.	Usamos JOIN para ver o pedido junto com o nome do cliente.
SUM e AVG	Calculadora do banco.	SUM soma valores; AVG calcula média.
ROW_NUMBER	Senha na fila do ranking.	Numera as linhas em uma ordem definida.
LAG	Olhar para a linha anterior.	Compara o mês atual com o mês passado.
Documentação	Manual do banco.	Explica como o banco foi pensado, criado e usado.

Dica de condução: sempre que um aluno travar no termo técnico, volte para a analogia. Depois pergunte: "No nosso banco, qual seria a gaveta? Qual seria a ficha? Qual seria a ponte entre as fichas?"

4. Normalização explicada de forma simples

Forma normal	Ideia principal	Exemplo simples
1NF	Cada campo deve ter apenas um valor. Sem listas dentro de uma célula.	Separar telefones do cliente em linhas ou tabela própria.
2NF	Cada informação deve depender da chave certa e estar na tabela certa.	Dados do cliente ficam em Cliente, não repetidos em Pedido.
3NF	Uma coluna não chave não deve depender de outra coluna não chave.	Se cidade depende do CEP, pode ser separada em tabela própria.
4NF	Separar listas independentes que estão misturadas na mesma tabela.	Funcionário com várias habilidades e vários projetos deve ter tabelas separadas.

5. Exemplos solucionados para a aula teórica

Exemplo 1 - Identificando entidades

Cenário: uma loja de roupas precisa controlar clientes, produtos e pedidos.

Entidade	Atributos possíveis	Por que existe?
Cliente	id_cliente, nome, email	Guardar quem compra.

Produto	id_produto, nome, preco	Guardar o que é vendido.
Pedido	id_pedido, id_cliente, data_pedido	Guardar uma compra feita por um cliente.
ItemPedido	id_item, id_pedido, id_produto, quantidade	Guardar os produtos dentro de cada pedido.

Explicação: entidade é tudo aquilo que precisa virar uma tabela ou parte importante do banco. Cliente, Produto e Pedido são assuntos centrais do sistema.

Exemplo 2 - Chave primária e chave estrangeira

```
CREATE TABLE Cliente (
    id_cliente INTEGER PRIMARY KEY,
    nome TEXT,
    email TEXT
);

CREATE TABLE Pedido (
    id_pedido INTEGER PRIMARY KEY,
    id_cliente INTEGER,
    data_pedido TEXT,
    valor_total REAL,
    FOREIGN KEY (id_cliente) REFERENCES Cliente(id_cliente)
);
```

Explicação: id_cliente é chave primária na tabela Cliente. Na tabela Pedido, id_cliente vira chave estrangeira para indicar qual cliente fez o pedido.

Exemplo 3 - Aplicando 1NF

Antes da 1NF:

id_cliente	nome	telefones
1	João	9999-1111, 9888-2222

Depois da 1NF:

Tabela	Campos
Cliente	id_cliente, nome
TelefoneCliente	id_cliente, telefone

Explicação: a lista de telefones saiu de uma única célula e foi separada em registros próprios.

Exemplo 4 - Funções de agregação

```
SELECT SUM(valor_total) AS total_vendas FROM Pedido;
SELECT AVG(valor_total) AS media_vendas FROM Pedido;
SELECT COUNT(*) AS quantidade_pedidos FROM Pedido;
```

SUM soma valores, AVG calcula média e COUNT conta registros.

Exemplo 5 - Funções analíticas

```
SELECT
    id_cliente,
    valor_total,
    ROW_NUMBER() OVER (ORDER BY valor_total DESC) AS ranking
FROM Pedido;
```

ROW_NUMBER cria uma numeração sequencial, útil para montar ranking. É como organizar uma fila do maior valor para o menor e colocar uma senha em cada posição.

Exemplo 6 - LAG

```
SELECT
    mes,
    total_vendas,
    LAG(total_vendas) OVER (ORDER BY mes) AS vendas_mes_anterior
FROM VendasMensais;
```

LAG olha para a linha anterior. É útil para comparar mês atual com mês anterior.

5.1 Mais exemplos de código SQL solucionados

Os exemplos abaixo usam uma base simples de loja de roupas. Eles foram pensados para funcionar em SQLite/OneCompiler e também podem ser adaptados para MySQL. O objetivo é mostrar o raciocínio passo a passo antes da atividade prática.

Exemplo 7 - Base completa em SQL: loja de roupas

Este exemplo é compatível com SQLite/OneCompiler e também pode ser adaptado para MySQL. Ele cria uma base pequena com Cliente, Produto e Pedido.

```
CREATE TABLE Cliente (
    id_cliente INTEGER PRIMARY KEY,
    nome TEXT NOT NULL,
    email TEXT
);

CREATE TABLE Produto (
    id_produto INTEGER PRIMARY KEY,
    nome TEXT NOT NULL,
    preco REAL NOT NULL
);

CREATE TABLE Pedido (
    id_pedido INTEGER PRIMARY KEY,
    id_cliente INTEGER NOT NULL,
    id_produto INTEGER NOT NULL,
    quantidade INTEGER NOT NULL,
    data_pedido TEXT,
    FOREIGN KEY (id_cliente) REFERENCES Cliente(id_cliente),
    FOREIGN KEY (id_produto) REFERENCES Produto(id_produto)
);
```

Analogia: primeiro construímos as gavetas do armário. Cliente, Produto e Pedido são gavetas diferentes, mas Pedido aponta para Cliente e Produto por meio das chaves estrangeiras.

Exemplo 8 - Inserindo dados simulados

Depois da estrutura pronta, inserimos dados para testar o banco.

```
INSERT INTO Cliente VALUES (1, 'João', 'joao@email.com');
INSERT INTO Cliente VALUES (2, 'Maria', 'maria@email.com');

INSERT INTO Produto VALUES (1, 'Camiseta', 50.00);
INSERT INTO Produto VALUES (2, 'Calça Jeans', 120.00);
INSERT INTO Produto VALUES (3, 'Tênis', 200.00);

INSERT INTO Pedido VALUES (1, 1, 1, 2, '2026-06-10');
```

```
INSERT INTO Pedido VALUES (2, 1, 3, 1, '2026-06-11');  
INSERT INTO Pedido VALUES (3, 2, 2, 1, '2026-06-12');
```

Analogia: agora estamos colocando fichas dentro das gavetas. Cada INSERT é uma ficha nova sendo guardada.

Exemplo 9 - Consulta com JOIN para juntar informações

O JOIN permite mostrar dados que estão separados em tabelas diferentes.

```
SELECT  
    Pedido.id_pedido,  
    Cliente.nome AS cliente,  
    Produto.nome AS produto,  
    Pedido.quantidade,  
    Produto.preco,  
    (Pedido.quantidade * Produto.preco) AS total  
FROM Pedido  
JOIN Cliente ON Pedido.id_cliente = Cliente.id_cliente  
JOIN Produto ON Pedido.id_produto = Produto.id_produto;
```

Analogia: o JOIN é como pegar a ficha do pedido e buscar, pelo número de referência, a ficha do cliente e a ficha do produto.

Exemplo 10 - Total de vendas com SUM

A função SUM soma valores. Aqui calculamos o total vendido.

```
SELECT  
    SUM(Pedido.quantidade * Produto.preco) AS total_vendido  
FROM Pedido  
JOIN Produto ON Pedido.id_produto = Produto.id_produto;
```

Analogia: SUM é a calculadora do caixa somando tudo que foi vendido.

Exemplo 11 - Média de valor dos pedidos com AVG

A função AVG calcula a média. Aqui calculamos o valor médio dos pedidos.

```
SELECT  
    AVG(Pedido.quantidade * Produto.preco) AS media_por_pedido  
FROM Pedido  
JOIN Produto ON Pedido.id_produto = Produto.id_produto;
```

Analogia: AVG pega todos os valores, soma e divide pela quantidade de pedidos, como calcular média de notas.

Exemplo 12 - Contagem de pedidos por cliente com COUNT e GROUP BY

COUNT conta registros. GROUP BY agrupa os pedidos por cliente.

```
SELECT  
    Cliente.nome,  
    COUNT(Pedido.id_pedido) AS quantidade_pedidos  
FROM Cliente  
JOIN Pedido ON Cliente.id_cliente = Pedido.id_cliente  
GROUP BY Cliente.nome;
```

Analogia: GROUP BY separa os pedidos em montinhos por cliente; COUNT conta quantos pedidos há em cada montinho.

Exemplo 13 - Ranking de pedidos com ROW_NUMBER

ROW_NUMBER cria uma numeração seguindo uma ordem definida.

```

SELECT
    Cliente.nome AS cliente,
    Produto.nome AS produto,
    (Pedido.quantidade * Produto.preco) AS total,
    ROW_NUMBER() OVER (
        ORDER BY (Pedido.quantidade * Produto.preco) DESC
    ) AS posicao
FROM Pedido
JOIN Cliente ON Pedido.id_cliente = Cliente.id_cliente
JOIN Produto ON Pedido.id_produto = Produto.id_produto;

```

Analogia: o banco organiza os pedidos do maior para o menor e entrega uma senha de posição para cada linha.

Exemplo 14 - Comparando venda atual com venda anterior usando LAG

LAG busca o valor da linha anterior. Ele ajuda a comparar períodos.

```

CREATE TABLE VendasMensais (
    mes TEXT PRIMARY KEY,
    total REAL
);

INSERT INTO VendasMensais VALUES ('Janeiro', 1000.00);
INSERT INTO VendasMensais VALUES ('Fevereiro', 1300.00);
INSERT INTO VendasMensais VALUES ('Março', 900.00);

SELECT
    mes,
    total,
    LAG(total) OVER (ORDER BY mes) AS total_mes_anterior,
    total - LAG(total) OVER (ORDER BY mes) AS diferenca
FROM VendasMensais;

```

Analogia: LAG é como olhar a nota anterior do aluno para comparar se ele melhorou ou piorou.

Exemplo 15 - Subconsulta para encontrar pedidos acima da média

A subconsulta permite usar o resultado de uma consulta dentro de outra.

```

SELECT
    Pedido.id_pedido,
    (Pedido.quantidade * Produto.preco) AS total
FROM Pedido
JOIN Produto ON Pedido.id_produto = Produto.id_produto
WHERE (Pedido.quantidade * Produto.preco) > (
    SELECT AVG(Pedido.quantidade * Produto.preco)
    FROM Pedido
    JOIN Produto ON Pedido.id_produto = Produto.id_produto
);

```

Analogia: primeiro o banco calcula a média da turma; depois mostra apenas quem ficou acima dessa média.

6. Roteiro da aula teórica - 50 minutos

Tempo	Momento	Condução sugerida
5 min	Abertura	Perguntar: o que pode acontecer se criarmos tabelas sem pensar antes na organização dos dados?
10 min	Modelagem e requisitos	Explicar tema, entidades, atributos, relacionamentos e regras de negócio usando analogia da planta da casa.
15 min	Normalização	Apresentar 1NF, 2NF, 3NF e 4NF com exemplos simples de clientes, telefones, pedidos e pagamentos.
8 min	UML e ER	Mostrar que diagramas são mapas do banco de dados e ajudam a equipe a pensar antes de programar.

9 min	SQL para criação e análise	Apresentar CREATE TABLE, INSERT, SELECT, SUM, AVG, COUNT, ROW_NUMBER e LAG.
3 min	Fechamento	Retomar: planejar, modelar, normalizar, criar, inserir, consultar e documentar.

7. Roteiro da aula prática - 50 minutos

Tempo	Etapa	O que os alunos fazem
5 min	Escolha do tema	Escolhem um tema simples: biblioteca, loja, eventos, clínica veterinária, lanchonete ou sistema escolar.
8 min	Requisitos	Respondem qual problema o sistema resolve, quais dados serão armazenados e quais regras existem.
10 min	Modelagem inicial	Criam uma tabela com entidades, atributos, chave primária e relacionamentos.
8 min	Normalização básica	Verificam listas, repetições e informações fora da tabela correta.
10 min	Criação em SQL	Criam pelo menos 3 tabelas com chave primária e pelo menos 1 chave estrangeira.
7 min	Inserção e consultas	Inserem dados simulados e fazem pelo menos 3 consultas SQL.
2 min	Registro final	Anotam tema, tabelas, dificuldade e aprendizado.

8. Atividade prática adaptada - Projeto de modelagem e desenvolvimento de banco de dados

A atividade foi adaptada para nível iniciante, mantendo o objetivo original: desenvolver um projeto de banco de dados com planejamento, modelagem, criação de esquema, inserção de dados, consultas e relatório.

Regras mínimas do projeto

- Escolher um tema simples.
- Criar no mínimo 3 tabelas.
- Criar no mínimo 3 atributos por tabela.
- Cada tabela deve ter uma chave primária.
- O projeto deve ter pelo menos 1 chave estrangeira.
- Inserir pelo menos 5 registros no total.
- Criar pelo menos 3 consultas SQL simples.
- Entregar um pequeno relatório explicando o projeto e os desafios encontrados.

Temas sugeridos

- Sistema de biblioteca
- Loja de roupas
- Plataforma de eventos
- Clínica veterinária
- Lanchonete
- Sistema escolar simples

Modelo para preencher - Planejamento

Item	Resposta do grupo
Tema escolhido	Sistema de Biblioteca
Problema que o sistema resolve	Organizar livros, autores e empréstimos
Quem usaria o sistema?	Bibliotecários
Quais dados precisam ser armazenados?	Livros, autores e empréstimos
Principais regras de negócio	Um livro possui um autor e pode ser emprestado para um usuário

Modelo para preencher - Entidades e relacionamentos

Entidade	Atributos	Chave primária	Relacionamento
Autores	id_autor, nome, nacionalidade	id_autor	Relaciona-se com Livros
Livros	id_livro, titulo, ano_publicacao, id_autor	id_livro	Possui um autor
Empréstimos	id_emprestimo, data_emprestimo, id_livro, id_usuario	id_emprestimo	Liga Livros e Usuários
Usuários	id_usuario, nome, telefone	id_usuario	Realiza empréstimos

Relacionamentos

- **Autor** → **Livro** = 1 para N (um autor pode escrever vários livros).
- **Livro** → **Empréstimo** = 1 para N (um livro pode ser emprestado várias vezes).
- **Usuário** → **Empréstimo** = 1 para N (um usuário pode fazer vários empréstimos).

Checklist de normalização básica

Pergunta	Sim/Não	Ajuste necessário
Existe alguma lista dentro de uma célula?	não	nenhum
Alguma informação aparece repetida em várias tabelas?	não	nenhum
Dados do cliente aparecem dentro da tabela de pedidos?	não	nenhum
Cada tabela tem uma chave primária?	sim	nenhum
Existe pelo menos uma chave estrangeira ligando tabelas?	sim	nenhum

Espaço para scripts SQL

Cole ou escreva aqui os comandos CREATE TABLE, INSERT e SELECT usados pelo grupo.

CREATE TABLE:

Código SQL

```
CREATE TABLE Autores (
  id_autor INT PRIMARY KEY,
  nome VARCHAR(100),
  nacionalidade VARCHAR(50)
);

CREATE TABLE Livros (
  id_livro INT PRIMARY KEY,
  titulo VARCHAR(100),
  ano_publicacao INT,
  id_autor INT,
  FOREIGN KEY (id_autor) REFERENCES Autores(id_autor)
);

CREATE TABLE Empréstimos (
  id_emprestimo INT PRIMARY KEY,
  nome_usuario VARCHAR(100),
  data_emprestimo DATE,
  id_livro INT,
  FOREIGN KEY (id_livro) REFERENCES Livros(id_livro)
);
```

INSERT:

Código SQL
<pre>CREATE TABLE Autores (id_autor INT PRIMARY KEY, nome VARCHAR(100), nacionalidade VARCHAR(50)); CREATE TABLE Livros (id_livro INT PRIMARY KEY, titulo VARCHAR(100), ano_publicacao INT, id_autor INT, FOREIGN KEY (id_autor) REFERENCES Autores(id_autor)); CREATE TABLE Emprestimos (id_emprestimo INT PRIMARY KEY, nome_usuario VARCHAR(100), data_emprestimo DATE, id_livro INT, FOREIGN KEY (id_livro) REFERENCES Livros(id_livro));</pre>

SELECT / Consultas:

Código SQL
<pre>1. Listar todos os livros cadastrados SELECT * FROM Livros; 2. Mostrar os livros e seus respectivos autores SELECT Livros.titulo, Autores.nome AS autor FROM Livros INNER JOIN Autores ON Livros.id_autor = Autores.id_autor; 3. Mostrar os empréstimos com o nome do usuário e o livro emprestado SELECT Usuarios.nome AS usuario, Livros.titulo AS livro, Emprestimos.data_emprestimo FROM Emprestimos INNER JOIN Usuarios ON Emprestimos.id_usuario = Usuarios.id_usuario INNER JOIN Livros ON Emprestimos.id_livro = Livros.id_livro;</pre>

Entrega final

- Nome dos integrantes.
- Tema escolhido.
- Requisitos do sistema.
- Entidades e atributos.
- Relacionamentos.
- Diagrama simples ou tabela de modelagem.
- Scripts SQL de criação das tabelas.
- Scripts SQL de inserção de dados.
- Pelo menos 3 consultas SQL.
- Breve explicação dos desafios encontrados.

9. Erros comuns

Erro comum	Como corrigir / orientar
Colocar todos os dados em uma única tabela.	Separar por assuntos: Cliente, Produto, Pedido, Pagamento etc.
Repetir nome e email do cliente em várias tabelas.	Guardar dados do cliente apenas na tabela Cliente e usar id_cliente nas outras.
Esquecer chave primária.	Cada tabela precisa de um identificador único.
Esquecer chave estrangeira.	Tabelas relacionadas precisam de uma ponte.
Confundir entidade com atributo.	Entidade é o assunto principal; atributo é uma característica desse assunto.
Achar que normalizar é apenas dividir tabelas.	Normalizar é dividir com lógica para reduzir repetição e manter consistência.
Não testar consultas.	Executar SELECT simples para verificar se os dados foram inseridos corretamente.

10. Critérios de avaliação

Critério	Valor sugerido
Participação e organização do grupo	1,0
Clareza do tema e dos requisitos	1,5
Identificação de entidades, atributos e relacionamentos	2,0
Aplicação básica da normalização	1,5
Criação correta das tabelas em SQL	2,0
Inserção de dados e consultas SQL funcionando	1,5
Relatório final e explicação dos desafios	0,5

Observação: os valores podem ser ajustados conforme a necessidade da turma.

11. Fechamento da aula

Finalize reforçando que banco de dados é planejamento + organização + implementação. O SQL é importante, mas a estrutura correta nasce antes do código, na análise do problema e na modelagem dos dados.